05. Fork Choice 알고리즘 구현 분석

개요

Fork Choice 알고리즘은 Ethereum 2.0 합의 프로토콜의 핵심 구성 요소로, 여러 개의 유효한 블록 체인이 존재할 때 어떤 체인을 "정식(canonical)" 체인으로 선택할지 결정하는 알고리즘입니다. Lighthouse는 이를 **LMD-GHOST (Latest Message Driven Greedy Heaviest Observed SubTree)** 알고리즘과 **ProtoArray** 데이터 구조를 통해 구현합니다.

이 문서에서는 `consensus/fork_choice` 와 `consensus/proto_array` 모듈의 상세한 구현을 분석하고, 실제 포크 선택 로직과 ProtoArray 데이터 구조의 동작 원리를 실제 함수명과 코드 예시를 통해 설명합니다.

Fork Choice 모듈 구조

1. 모듈 계층 구조

```
consensus/fork_choice/
├── src/
| ├── lib.rs # 모듈 진입점 및 공개 API
| ├── fork_choice.rs # 메인 ForkChoice 구조체
| ├── fork_choice_store.rs # 상태 저장소 인터페이스
| └── metrics.rs # 성능 메트릭
```

```
consensus/proto_array/
├── src/
```

```
│ ├── lib.rs # 모듈 진입점
│ ├── proto_array.rs # 핵심 ProtoArray 구조체
│ ├── proto_array_fork_choice.rs # ProtoArrayForkChoice 래퍼
│ ├── justified_balances.rs # 검증자 잔액 관리
│ ├── error.rs # 에러 타입 정의
│ └── ssz_container.rs # 직렬화 컨테이너
```

2. 핵심 구조체 관계도

```
┌─────────────────────────────────────────────────────────────┐
│ ForkChoice<T, E> │
├─────────────────────────────────────────────────────────────┤
│ - fc_store: T │
│ - proto_array: ProtoArrayForkChoice │
│ - queued_attestations: Vec<QueuedAttestation> │
│ - forkchoice_update_parameters: ForkchoiceUpdateParameters │
└─────────────────────────────────────────────────────────────┘
│
▼
┌─────────────────────────────────────────────────────────────┐
│ ProtoArrayForkChoice │
├─────────────────────────────────────────────────────────────┤
│ - proto_array: ProtoArray │
```

```
| - votes: ElasticList<VoteTracker> |
| - balances: JustifiedBalances |
└──────────────────────────────────────────────────────────────┘
|
▼
┌──────────────────────────────────────────────────────────────┐
| ProtoArray |
├──────────────────────────────────────────────────────────────┤
| - nodes: Vec<ProtoNode> |
| - indices: HashMap<Hash256, usize> |
| - justified_checkpoint: Checkpoint |
| - finalized_checkpoint: Checkpoint |
| - previous_proposer_boost: ProposerBoost |
└──────────────────────────────────────────────────────────────┘
```##
ProtoArray 데이터 구조 분석

1. ProtoNode 구조체

ProtoArray의 핵심은 각 블록을 나타내는 `ProtoNode` 구조체입니다:

```rust
// consensus/proto_array/src/proto_array.rs
```

```
pub struct ProtoNode {
/// 블록의 슬롯 번호
pub slot: Slot,
/// 상태 루트 해시
pub state_root: Hash256,
/// 타겟 루트 (증명에서 사용)
pub target_root: Hash256,
/// 현재 에포크 셔플링 ID
pub current_epoch_shuffling_id: AttestationShufflingId,
/// 다음 에포크 셔플링 ID
pub next_epoch_shuffling_id: AttestationShufflingId,
/// 블록 루트 해시
pub root: Hash256,
/// 부모 노드의 인덱스 (Option<usize>)
pub parent: Option<usize>,
/// 정당화된 체크포인트
pub justified_checkpoint: Checkpoint,
/// 최종화된 체크포인트
pub finalized_checkpoint: Checkpoint,
/// 누적된 투표 가중치
pub weight: u64,
/// 최적의 자식 노드 인덱스
pub best_child: Option<usize>,
```

```
/// 최적의 후손 노드 인덱스
pub best_descendant: Option<usize>,
/// 실행 페이로드 상태
pub execution_status: ExecutionStatus,
/// 미실현 정당화 체크포인트
pub unrealized_justified_checkpoint: Option<Checkpoint>,
/// 미실현 최종화 체크포인트
pub unrealized_finalized_checkpoint: Option<Checkpoint>,
}
```

2. ProtoArray 구조체

```
// consensus/proto_array/src/proto_array.rs
pub struct ProtoArray {
/// 프루닝 임계값 (노드 수)
pub prune_threshold: usize,
/// 현재 정당화된 체크포인트
pub justified_checkpoint: Checkpoint,
/// 현재 최종화된 체크포인트
pub finalized_checkpoint: Checkpoint,
/// 모든 블록 노드들의 벡터
pub nodes: Vec<ProtoNode>,
/// 블록 해시 → 노드 인덱스 매핑
```

```
pub  indices: HashMap<Hash256, usize>,

/// 이전 제안자 부스트 정보

pub  previous_proposer_boost: ProposerBoost,

}
```

3. 투표 추적 시스템

```
// consensus/proto_array/src/proto_array_fork_choice.rs

pub  struct VoteTracker {

/// 현재 투표한 블록 루트

current_root: Hash256,

/// 다음에 투표할 블록 루트

next_root: Hash256,

/// 다음 투표의 타겟 에포크

next_epoch: Epoch,

}

```##

Fork Choice 알고리즘 핵심 로직

1. 헤드 블록 선택 과정 (`get_head`)

Fork Choice의 핵심 함수인 `get_head`는 다음과 같은 단계로 동작합니다:
```
```

```rust
// consensus/fork_choice/src/fork_choice.rs
impl<T, E> ForkChoice<T, E>
where
T: ForkChoiceStore<E>,
E: EthSpec,
{
pub fn get_head(
&mut self,
system_time_current_slot: Slot,
spec: &ChainSpec,
) -> Result<Hash256, Error<T::Error>> {
// 1. 현재 슬롯 업데이트
let current_slot = self.update_time(system_time_current_slot)?;

let store = &mut self.fc_store;

// 2. ProtoArray에서 헤드 찾기
let head_root = self.proto_array.find_head::<E>(
*store.justified_checkpoint(),
*store.finalized_checkpoint(),
store.justified_balances(),
store.proposer_boost_root(),
```

```
store.equivocating_indices(),
current_slot,
spec,
)?;


// 3. 실행 레이어를 위한 캐시 업데이트
let  head_hash = self
.get_block(&head_root)
.and_then(|b| b.execution_status.block_hash());
let  justified_root = self.justified_checkpoint().root;
let  finalized_root = self.finalized_checkpoint().root;
let  justified_hash = self
.get_block(&justified_root)
.and_then(|b| b.execution_status.block_hash());
let  finalized_hash = self
.get_block(&finalized_root)
.and_then(|b| b.execution_status.block_hash());
self.forkchoice_update_parameters = ForkchoiceUpdateParameters {
head_root,
head_hash,
justified_hash,
finalized_hash,
};
```

```
Ok(head_root)
}
}
```

2. ProtoArray의 헤드 찾기 (`find_head`)

```
// consensus/proto_array/src/proto_array_fork_choice.rs
impl ProtoArrayForkChoice {
pub fn find_head<E: EthSpec>(
&mut self,
justified_checkpoint: Checkpoint,
finalized_checkpoint: Checkpoint,
justified_state_balances: &JustifiedBalances,
proposer_boost_root: Hash256,
equivocating_indices: &BTreeSet<u64>,
current_slot: Slot,
spec: &ChainSpec,
) -> Result<Hash256, String> {
let old_balances = &mut self.balances;
let new_balances = justified_state_balances;

// 1. 투표 변화량(델타) 계산
```

```
let  deltas = compute_deltas(
&self.proto_array.indices,
&mut  self.votes,
&old_balances.effective_balances,
&new_balances.effective_balances,
equivocating_indices,
)?;


// 2. 점수 변화 적용
self.proto_array.apply_score_changes::<E>(
deltas,
justified_checkpoint,
finalized_checkpoint,
new_balances,
proposer_boost_root,
current_slot,
spec,
)?;


// 3. 잔액 업데이트
*old_balances = new_balances.clone();


// 4. 최종 헤드 찾기
```

```
self.proto_array.find_head::<E>(
&justified_checkpoint.root,
current_slot
)
}
}
```### 3. 투표
델타 계산 (`compute_deltas`)

이 함수는 검증자들의 투표 변화를 계산하여 각 블록의 가중치 변화량을 산출합니다:

```rust
// consensus/proto_array/src/proto_array_fork_choice.rs
fn compute_deltas(
indices: &HashMap<Hash256, usize>,
votes: &mut ElasticList<VoteTracker>,
old_balances: &[u64],
new_balances: &[u64],
equivocating_indices: &BTreeSet<u64>,
) -> Result<Vec<i64>, Error> {
let mut deltas = vec![0_i64; indices.len()];

for (val_index, vote) in votes.iter_mut().enumerate() {
```

```
// 제네시스 블록에 대한 투표는 무시
if vote.current_root == Hash256::zero() && vote.next_root == Hash256::zero() {
continue;
}

// 슬래싱된 검증자 처리
if equivocating_indices.contains(&(val_index as u64)) {
if !vote.current_root.is_zero() {
let old_balance = old_balances.get(val_index).copied().unwrap_or(0);

if let Some(current_delta_index) = indices.get(&vote.current_root).copied() {
let delta = deltas[current_delta_index]
.checked_sub(old_balance as i64)
.ok_or(Error::DeltaOverflow(current_delta_index))?;
deltas[current_delta_index] = delta;
}

vote.current_root = Hash256::zero();
}
continue;
}

let old_balance = old_balances.get(val_index).copied().unwrap_or(0);
```

```
let new_balance = new_balances.get(val_index).copied().unwrap_or(0);

// 투표가 변경되었거나 잔액이 변경된 경우
if vote.current_root != vote.next_root || old_balance != new_balance {
// 이전 투표에서 가중치 제거
if let Some(current_delta_index) = indices.get(&vote.current_root).copied() {
let delta = deltas[current_delta_index]
.checked_sub(old_balance as i64)
.ok_or(Error::DeltaOverflow(current_delta_index))?;
deltas[current_delta_index] = delta;
}

// 새 투표에 가중치 추가
if let Some(next_delta_index) = indices.get(&vote.next_root).copied() {
let delta = deltas[next_delta_index]
.checked_add(new_balance as i64)
.ok_or(Error::DeltaOverflow(next_delta_index))?;
deltas[next_delta_index] = delta;
}

vote.current_root = vote.next_root;
}
}
```

```
Ok(deltas)

}
```

4. 점수 변화 적용 (`apply_score_changes`)

계산된 델타를 실제 노드들에 적용하고 best_child/best_descendant를 업데이트합니다:

```
// consensus/proto_array/src/proto_array.rs

impl ProtoArray {

pub  fn  apply_score_changes<E: EthSpec>(

&mut  self,

mut  deltas: Vec<i64>,

justified_checkpoint: Checkpoint,

finalized_checkpoint: Checkpoint,

new_justified_balances: &JustifiedBalances,

proposer_boost_root: Hash256,

current_slot: Slot,

spec: &ChainSpec,

) -> Result<(), Error> {

// 체크포인트 업데이트

if  justified_checkpoint != self.justified_checkpoint

|| finalized_checkpoint != self.finalized_checkpoint

{
```

```
self.justified_checkpoint = justified_checkpoint;
self.finalized_checkpoint = finalized_checkpoint;
}

let mut proposer_score = 0;

// 1단계: 역순으로 노드들을 순회하며 가중치 업데이트
for node_index in (0..self.nodes.len()).rev() {
let node = self.nodes.get_mut(node_index)
.ok_or(Error::InvalidNodeIndex(node_index))?;

if node.root == Hash256::zero() {
continue;
}

let execution_status_is_invalid = node.execution_status.is_invalid();

let mut node_delta = if execution_status_is_invalid {
// 무효한 실행 페이로드를 가진 노드는 가중치를 0으로 설정
0_i64.checked_sub(node.weight as i64)
.ok_or(Error::InvalidExecutionDeltaOverflow(node_index))?
} else {
deltas.get(node_index).copied()
```

```
.ok_or(Error::InvalidNodeDelta(node_index))?
};

// 이전 제안자 부스트 제거
if  self.previous_proposer_boost.root != Hash256::zero()
&& self.previous_proposer_boost.root == node.root
&& !execution_status_is_invalid
{
node_delta = node_delta
.checked_sub(self.previous_proposer_boost.score as i64)
.ok_or(Error::DeltaOverflow(node_index))?;
}

// 새 제안자 부스트 적용
if  let Some(proposer_score_boost) = spec.proposer_score_boost {
if  proposer_boost_root != Hash256::zero()
&& proposer_boost_root == node.root
&& !execution_status_is_invalid
{
proposer_score = calculate_committee_fraction::<E>(
new_justified_balances,
proposer_score_boost,
).ok_or(Error::ProposerBoostOverflow(node_index))?;
```

```
node_delta = node_delta
.checked_add(proposer_score  as i64)
.ok_or(Error::DeltaOverflow(node_index))?;
}
}

// 노드에 델타 적용
if  execution_status_is_invalid {
node.weight = 0;
} else  if  node_delta < 0 {
node.weight = node.weight
.checked_sub(node_delta.unsigned_abs())
.ok_or(Error::DeltaOverflow(node_index))?;
} else {
node.weight = node.weight
.checked_add(node_delta  as u64)
.ok_or(Error::DeltaOverflow(node_index))?;
}

// 부모 노드의 델타에 현재 노드의 델타 전파
if  let Some(parent_index) = node.parent {
let  parent_delta = deltas.get_mut(parent_index)
.ok_or(Error::InvalidParentDelta(parent_index))?;
```

```
*parent_delta += node_delta;
}
}

// 제안자 부스트 업데이트
self.previous_proposer_boost = ProposerBoost {
root: proposer_boost_root,
score: proposer_score,
};

// 2단계: best_child와 best_descendant 업데이트
for node_index in (0..self.nodes.len()).rev() {
let node = self.nodes.get(node_index)
.ok_or(Error::InvalidNodeIndex(node_index))?;

if let Some(parent_index) = node.parent {
self.maybe_update_best_child_and_descendant::<E>(
parent_index,
node_index,
current_slot,
)?;
}
}
```

```
Ok(())
}
}
```### 5
. 최종 헤드 결정 (`find_head`)

```rust
// consensus/proto_array/src/proto_array.rs
impl ProtoArray {
pub fn find_head<E: EthSpec>(
&self,
justified_root: &Hash256,
current_slot: Slot,
) -> Result<Hash256, Error> {
// 정당화된 블록 찾기
let justified_index = self.indices.get(justified_root).copied()
.ok_or(Error::JustifiedNodeUnknown(*justified_root))?;

let justified_node = self.nodes.get(justified_index)
.ok_or(Error::InvalidJustifiedIndex(justified_index))?;

// 무효한 실행 상태를 가진 정당화된 블록은 에러
```

```
if  justified_node.execution_status.is_invalid() {
return Err(Error::InvalidJustifiedCheckpointExecutionStatus {
justified_root: *justified_root,
});
}

// 최적의 후손 찾기 (없으면 자기 자신)
let  best_descendant_index = justified_node.best_descendant
.unwrap_or(justified_index);

let  best_node = self.nodes.get(best_descendant_index)
.ok_or(Error::InvalidBestDescendant(best_descendant_index))?;

// 헤드로 유효한지 검증
if !self.node_is_viable_for_head::<E>(best_node, current_slot) {
return Err(Error::InvalidBestNode(Box::new(InvalidBestNodeInfo {
current_slot,
start_root: *justified_root,
justified_checkpoint: self.justified_checkpoint,
finalized_checkpoint: self.finalized_checkpoint,
head_root: best_node.root,
head_justified_checkpoint: best_node.justified_checkpoint,
head_finalized_checkpoint: best_node.finalized_checkpoint,
```

```
}));
}

Ok(best_node.root)
}
}
```

블록 추가 과정 (on_block)

새로운 블록이 도착했을 때의 처리 과정:

1. Fork Choice 레벨에서의 블록 처리

```
// consensus/fork_choice/src/fork_choice.rs
impl<T, E> ForkChoice<T, E>
where
T: ForkChoiceStore<E>,
E: EthSpec,
{
pub fn on_block<Payload: AbstractExecPayload<E>>(
&mut self,
system_time_current_slot: Slot,
block: BeaconBlockRef<E, Payload>,
block_root: Hash256,
```

```
block_delay: Duration,
state: &BeaconState<E>,
payload_verification_status: PayloadVerificationStatus,
spec: &ChainSpec,
) -> Result<(), Error<T::Error>> {
// 이미 처리된 블록인지 확인
if  self.proto_array.contains_block(&block_root) {
return Ok(());
}

let  current_slot = self.update_time(system_time_current_slot)?;

// 부모 블록 존재 확인
let  parent_block = self.proto_array.get_block(&block.parent_root())
.ok_or_else(|| Error::InvalidBlock(InvalidBlock::UnknownParent(block.parent_root

// 미래 블록 검증
if  block.slot() > current_slot {
return Err(Error::InvalidBlock(InvalidBlock::FutureSlot {
current_slot,
block_slot: block.slot(),
}));
}
```

```
// 최종화 슬롯 이후 블록인지 확인
let finalized_slot = compute_start_slot_at_epoch::<E>(
self.fc_store.finalized_checkpoint().epoch
);
if block.slot() <= finalized_slot {
return Err(Error::InvalidBlock(InvalidBlock::FinalizedSlot {
finalized_slot,
block_slot: block.slot(),
}));
}

// 최종화된 블록의 후손인지 확인
let block_ancestor = self.get_ancestor(block.parent_root(), finalized_slot)?;
let finalized_root = self.fc_store.finalized_checkpoint().root;
if block_ancestor != Some(finalized_root) {
return Err(Error::InvalidBlock(InvalidBlock::NotFinalizedDescendant {
finalized_root,
block_ancestor,
}));
}

// 제안자 부스트 적용 (적시 블록인 경우)
```

```
let  is_before_attesting_interval = block_delay < Duration::from_secs(
spec.seconds_per_slot / INTERVALS_PER_SLOT
);
let  is_first_block = self.fc_store.proposer_boost_root().is_zero();
if  current_slot == block.slot() && is_before_attesting_interval && is_first_blo
self.fc_store.set_proposer_boost_root(block_root);
}


// 체크포인트 업데이트
self.update_checkpoints(
state.current_justified_checkpoint(),
state.finalized_checkpoint(),
)?;


// ProtoArray에 블록 추가
let  proto_block = Block {
slot: block.slot(),
root: block_root,
parent_root: Some(block.parent_root()),
state_root: block.state_root(),
target_root: self.calculate_target_root(block)?,
current_epoch_shuffling_id: self.get_current_shuffling_id(state)?,
next_epoch_shuffling_id: self.get_next_shuffling_id(state)?,
```

```
justified_checkpoint: state.current_justified_checkpoint(),
finalized_checkpoint: state.finalized_checkpoint(),
execution_status: self.determine_execution_status(payload_verification_status),
unrealized_justified_checkpoint: Some(unrealized_justified_checkpoint),
unrealized_finalized_checkpoint: Some(unrealized_finalized_checkpoint),
};

self.proto_array.process_block::<E>(proto_block, current_slot)
.map_err(Error::ProtoArrayStringError)?;

Ok(())
}
}
```#
2. ProtoArray 레벨에서의 블록 추가

```rust
// consensus/proto_array/src/proto_array.rs
impl ProtoArray {
pub fn on_block<E: EthSpec>(
&mut self,
block: Block,
current_slot: Slot
```

```
) -> Result<(), Error> {
// 이미 알려진 블록인지 확인
if  self.indices.contains_key(&block.root) {
return Ok(());
}

let  node_index = self.nodes.len();

// 새 ProtoNode 생성
let  node = ProtoNode {
slot: block.slot,
root: block.root,
target_root: block.target_root,
current_epoch_shuffling_id: block.current_epoch_shuffling_id,
next_epoch_shuffling_id: block.next_epoch_shuffling_id,
state_root: block.state_root,
parent: block.parent_root
.and_then(|parent| self.indices.get(&parent).copied()),
justified_checkpoint: block.justified_checkpoint,
finalized_checkpoint: block.finalized_checkpoint,
weight: 0,
best_child: None,
best_descendant: None,
```

```
execution_status: block.execution_status,
unrealized_justified_checkpoint: block.unrealized_justified_checkpoint,
unrealized_finalized_checkpoint: block.unrealized_finalized_checkpoint,
};

// 부모가 무효한 실행 상태를 가지는지 확인
if  let Some(parent_index) = node.parent {
let  parent = self.nodes.get(parent_index)
.ok_or(Error::InvalidNodeIndex(parent_index))?;
if  parent.execution_status.is_invalid() {
return Err(Error::ParentExecutionStatusIsInvalid {
block_root: block.root,
parent_root: parent.root,
});
}
}

// 노드 추가
self.indices.insert(node.root, node_index);
self.nodes.push(node.clone());

// 부모의 best_child/best_descendant 업데이트
if  let Some(parent_index) = node.parent {
```

```
self.maybe_update_best_child_and_descendant::<E>(
parent_index,
node_index,
current_slot,
)?;

// 유효한 실행 페이로드인 경우 조상들에게 전파
if matches!(block.execution_status, ExecutionStatus::Valid(_)) {
self.propagate_execution_payload_validation_by_index(parent_index)?;
}
}

Ok(())
}
}
```

증명 처리 (process_attestation)

검증자의 증명을 처리하여 투표를 업데이트합니다:

```
// consensus/proto_array/src/proto_array_fork_choice.rs
impl ProtoArrayForkChoice {
pub fn process_attestation(
```

```
&mut self,
validator_index: usize,
block_root: Hash256,
target_epoch: Epoch,
) -> Result<(), String> {
let vote = self.votes.get_mut(validator_index);


// 더 새로운 에포크의 투표이거나 첫 투표인 경우에만 업데이트
if target_epoch > vote.next_epoch || *vote == VoteTracker::default() {
vote.next_root = block_root;
vote.next_epoch = target_epoch;
}


Ok(())
}
}
```

실행 페이로드 상태 관리

1. ExecutionStatus 열거형

```
// consensus/proto_array/src/proto_array_fork_choice.rs
#[derive(Clone, Copy, Debug, PartialEq, Encode, Decode, Serialize, Deserialize)]
```

```rust
pub enum ExecutionStatus {
/// 실행 레이어에서 유효하다고 확인된 페이로드
Valid(ExecutionBlockHash),
/// 실행 레이어에서 무효하다고 확인된 페이로드
Invalid(ExecutionBlockHash),
/// 아직 실행 레이어에서 검증되지 않은 페이로드 (낙관적)
Optimistic(ExecutionBlockHash),
/// 머지 이전 블록이거나 실행과 무관한 블록
Irrelevant(bool),
}
```

2. 유효한 페이로드 전파

```rust
// consensus/proto_array/src/proto_array.rs
impl ProtoArray {
pub fn propagate_execution_payload_validation(
&mut self,
block_root: Hash256,
) -> Result<(), Error> {
let index = *self.indices.get(&block_root)
.ok_or(Error::NodeUnknown(block_root))?;
self.propagate_execution_payload_validation_by_index(index)
}
```

```
fn propagate_execution_payload_validation_by_index(
&mut self,
verified_node_index: usize,
) -> Result<(), Error> {
let mut index = verified_node_index;
loop {
let node = self.nodes.get_mut(index)
.ok_or(Error::InvalidNodeIndex(index))?;
let parent_index = match node.execution_status {
// 이미 유효한 노드에 도달하면 중단
ExecutionStatus::Valid(_) => return Ok(()),
// 무관한 노드에 도달하면 중단
ExecutionStatus::Irrelevant(_) => return Ok(()),
// 낙관적 상태를 유효로 변경
ExecutionStatus::Optimistic(payload_block_hash) => {
node.execution_status = ExecutionStatus::Valid(payload_block_hash);
if let Some(parent_index) = node.parent {
parent_index
} else {
return Ok(());
}
}
```

```
// 무효한 조상이 있으면 심각한 오류
ExecutionStatus::Invalid(ancestor_payload_block_hash) => {
return Err(Error::InvalidAncestorOfValidPayload {
ancestor_block_root: node.root,
ancestor_payload_block_hash,
});
}
};

index = parent_index;
}
}
}
```##
제안자 재구성 (Proposer Re-org)

Lighthouse는 특정 조건 하에서 제안자가 현재 헤드를 재구성할 수 있는 기능을 제공합니다:

1. 재구성 헤드 결정

```rust
// consensus/proto_array/src/proto_array_fork_choice.rs
impl ProtoArrayForkChoice {
```

```
pub fn get_proposer_head<E: EthSpec>(
&self,
current_slot: Slot,
canonical_head: Hash256,
justified_balances: &JustifiedBalances,
re_org_head_threshold: ReOrgThreshold,
re_org_parent_threshold: ReOrgThreshold,
disallowed_offsets: &DisallowedReOrgOffsets,
max_epochs_since_finalization: Epoch,
) -> Result<ProposerHeadInfo, ProposerHeadError<Error>> {
let info = self.get_proposer_head_info::<E>(
current_slot,
canonical_head,
justified_balances,
re_org_head_threshold,
re_org_parent_threshold,
disallowed_offsets,
max_epochs_since_finalization,
)?;


// 단일 슬롯만 재구성 허용
let head_slot_ok = info.head_node.slot + 1 == current_slot;
if !head_slot_ok {
```

```
return Err(DoNotReOrg::HeadDistance.into());
}

// 헤드의 가중치가 임계값보다 낮아야 함
let  head_weight = info.head_node.weight;
let  re_org_head_weight_threshold = info.re_org_head_weight_threshold;
let  weak_head = head_weight < re_org_head_weight_threshold;
if !weak_head {
return Err(DoNotReOrg::HeadNotWeak {
head_weight,
re_org_head_weight_threshold,
}.into());
}

// 부모의 가중치가 임계값보다 높아야 함
let  parent_weight = info.parent_node.weight;
let  re_org_parent_weight_threshold = info.re_org_parent_weight_threshold;
let  parent_strong = parent_weight > re_org_parent_weight_threshold;
if !parent_strong {
return Err(DoNotReOrg::ParentNotStrong {
parent_weight,
re_org_parent_weight_threshold,
}.into());
```

```
}

Ok(info)
}
}
```

2. 재구성 조건 검사

```
// consensus/proto_array/src/proto_array_fork_choice.rs
impl ProtoArrayForkChoice {
pub fn get_proposer_head_info<E: EthSpec>(
&self,
current_slot: Slot,
canonical_head: Hash256,
justified_balances: &JustifiedBalances,
re_org_head_threshold: ReOrgThreshold,
re_org_parent_threshold: ReOrgThreshold,
disallowed_offsets: &DisallowedReOrgOffsets,
max_epochs_since_finalization: Epoch,
) -> Result<ProposerHeadInfo, ProposerHeadError<Error>> {
// 헤드와 부모 노드 가져오기
let mut nodes = self.proto_array.iter_nodes(&canonical_head)
.take(2)
```

```
.cloned()
.collect::<Vec<_>>();

let parent_node = nodes.pop().ok_or(DoNotReOrg::MissingHeadOrParentNode)?;
let head_node = nodes.pop().ok_or(DoNotReOrg::MissingHeadOrParentNode)?;

let parent_slot = parent_node.slot;
let head_slot = head_node.slot;
let re_org_block_slot = head_slot + 1;

// 최종화 거리 확인
let proposal_epoch = re_org_block_slot.epoch(E::slots_per_epoch());
let finalized_epoch = head_node.unrealized_finalized_checkpoint
.ok_or(DoNotReOrg::MissingHeadFinalizedCheckpoint)?
.epoch;
let epochs_since_finalization = proposal_epoch.saturating_sub(finalized_epoch).
if epochs_since_finalization > max_epochs_since_finalization.as_u64() {
return Err(DoNotReOrg::ChainNotFinalizing {
epochs_since_finalization,
}.into());
}

// 부모 거리 확인
```

```
let  parent_slot_ok = parent_slot + 1 == head_slot;
if !parent_slot_ok {
return Err(DoNotReOrg::ParentDistance.into());
}

// 셔플링 안정성 확인
let  shuffling_stable = re_org_block_slot % E::slots_per_epoch() != 0;
if !shuffling_stable {
return Err(DoNotReOrg::ShufflingUnstable.into());
}

// 허용된 슬롯 오프셋 확인
let  offset = (re_org_block_slot % E::slots_per_epoch()).as_u64();
if  disallowed_offsets.offsets.contains(&offset) {
return Err(DoNotReOrg::DisallowedOffset { offset }.into());
}

// FFG 경쟁력 확인
let  ffg_competitive = parent_node.unrealized_justified_checkpoint
== head_node.unrealized_justified_checkpoint
&& parent_node.unrealized_finalized_checkpoint
== head_node.unrealized_finalized_checkpoint;
if !ffg_competitive {
```

```
return Err(DoNotReOrg::JustificationAndFinalizationNotCompetitive.into());
}

// 재구성 가중치 임계값 계산
let  re_org_head_weight_threshold = calculate_committee_fraction::<E>(
justified_balances,
re_org_head_threshold.0
).ok_or(Error::ReOrgThresholdOverflow)?;

let  re_org_parent_weight_threshold = calculate_committee_fraction::<E>(
justified_balances,
re_org_parent_threshold.0
).ok_or(Error::ReOrgThresholdOverflow)?;

Ok(ProposerHeadInfo {
head_node,
parent_node,
re_org_head_weight_threshold,
re_org_parent_weight_threshold,
current_slot,
})
}
}
```

```## 성

능 최적화 기법

### 1. 제안자 부스트 (Proposer Boost)

적시에 도착한 블록에 추가 가중치를 부여하여 네트워크 지연에 대한 저항성을 높입니다:

```rust
// consensus/proto_array/src/proto_array.rs
impl ProtoArray {
// apply_score_changes 함수 내에서
if let Some(proposer_score_boost) = spec.proposer_score_boost {
if proposer_boost_root != Hash256::zero()
&& proposer_boost_root == node.root
&& !execution_status_is_invalid
{
proposer_score = calculate_committee_fraction::<E>(
new_justified_balances,
proposer_score_boost,
).ok_or(Error::ProposerBoostOverflow(node_index))?;
node_delta = node_delta
.checked_add(proposer_score as i64)
.ok_or(Error::DeltaOverflow(node_index))?;
```
```

```
}
}
}

// 위원회 비율 계산
pub fn calculate_committee_fraction<E: EthSpec>(
justified_balances: &JustifiedBalances,
percentage: u64,
) -> Option<u64> {
let total_balance = justified_balances.total_effective_balance;
let committee_balance = total_balance / E::slots_per_epoch();
committee_balance.checked_mul(percentage)?.checked_div(100)
}
```

2. 엘라스틱 리스트 (Elastic List)

검증자 투표를 효율적으로 관리하기 위한 자동 확장 벡터:

```
// consensus/proto_array/src/proto_array_fork_choice.rs
#[derive(Default, Clone, Debug, PartialEq)]
pub struct ElasticList<T>(pub Vec<T>);

impl<T> ElasticList<T>
where
```

```
T: Default,
{
fn ensure(&mut self, i: usize) {
if self.0.len() <= i {
self.0.resize_with(i + 1, Default::default);
}
}

pub fn get_mut(&mut self, i: usize) -> &mut T {
self.ensure(i);
&mut self.0[i]
}

pub fn iter_mut(&mut self) -> impl Iterator<Item = &mut T> {
self.0.iter_mut()
}
}
```

3. 배치 처리

여러 증명을 한 번에 처리하여 성능을 향상시킵니다:

```
// consensus/fork_choice/src/fork_choice.rs
impl<T, E> ForkChoice<T, E>
```

```
where
T: ForkChoiceStore<E>,
E: EthSpec,
{
pub fn process_attestation(
&mut self,
attestation: IndexedAttestationRef<E>,
is_from_block: AttestationFromBlock,
) -> Result<(), Error<T::Error>> {
// 현재 슬롯의 증명은 큐에 저장
if attestation.data().slot == self.fc_store.get_current_slot() {
self.queued_attestations.push(QueuedAttestation::from(attestation));
return Ok(());
}

// 이전 슬롯의 증명은 즉시 처리
self.process_attestation_internal(attestation, is_from_block)
}

fn update_time(&mut self, current_slot: Slot) -> Result<Slot, Error<T::Error>>
let store_slot = self.fc_store.get_current_slot();
if current_slot > store_slot {
// 큐에 저장된 증명들 처리
```

```
let dequeued_attestations = dequeue_attestations(
current_slot,
&mut self.queued_attestations,
);


for queued_attestation in dequeued_attestations {
self.process_queued_attestation(queued_attestation)?;
}
}


self.fc_store.set_current_slot(current_slot);
Ok(current_slot)
}
}
```

프루닝 (Pruning)

오래된 블록들을 제거하여 메모리 사용량을 관리합니다:

```
// consensus/proto_array/src/proto_array.rs
impl ProtoArray {
pub fn maybe_prune(&mut self, finalized_root: Hash256) -> Result<(), Error> {
let finalized_index = *self.indices.get(&finalized_root)
.ok_or(Error::FinalizedNodeUnknown(finalized_root))?;
```

```
// 프루닝 임계값 미만이면 프루닝하지 않음
if finalized_index < self.prune_threshold {
return Ok(());
}

// 최종화된 블록 이전의 모든 블록들 제거
let removed_indices = self.nodes.drain(0..finalized_index).collect::<Vec<_>>();

// 인덱스 맵 업데이트
for removed_node in removed_indices {
self.indices.remove(&removed_node.root);
}

// 남은 노드들의 인덱스 조정
for (_, index) in self.indices.iter_mut() {
*index = index.saturating_sub(finalized_index);
}

// 부모 인덱스들 조정
for node in &mut self.nodes {
if let Some(parent_index) = node.parent {
if parent_index >= finalized_index {
```

```
node.parent = Some(parent_index - finalized_index);
} else {
node.parent = None;
}
}
if  let Some(best_child) = node.best_child {
if  best_child >= finalized_index {
node.best_child = Some(best_child - finalized_index);
} else {
node.best_child = None;
}
}
if  let Some(best_descendant) = node.best_descendant {
if  best_descendant >= finalized_index {
node.best_descendant = Some(best_descendant - finalized_index);
} else {
node.best_descendant = None;
}
}
}

Ok(())
}
```

```
}
```## 에러 처리 및
복구

1. 주요 에러 타입들

```rust
// consensus/fork_choice/src/fork_choice.rs
#[derive(Debug)]
pub  enum Error<T> {
InvalidAttestation(InvalidAttestation),
InvalidBlock(InvalidBlock),
ProtoArrayError(proto_array::Error),
MissingProtoArrayBlock(Hash256),
UnknownAncestor {
ancestor_slot: Slot,
descendant_root: Hash256,
},
BeaconStateError(BeaconStateError),
ForkChoiceStoreError(T),
InvalidPayloadStatus {
block_slot: Slot,
block_root: Hash256,
```

```
payload_verification_status: PayloadVerificationStatus,
},
}


// consensus/proto_array/src/error.rs
#[derive(Debug, PartialEq)]
pub  enum Error {
InvalidNodeIndex(usize),
InvalidParentIndex(usize),
InvalidBestChildIndex(usize),
InvalidBestDescendantIndex(usize),
InvalidJustifiedIndex(usize),
InvalidBestDescendant(usize),
InvalidParentDelta(usize),
InvalidNodeDelta(usize),
DeltaOverflow(usize),
ProposerBoostOverflow(usize),
ReOrgThresholdOverflow,
InvalidDeltaLen { deltas: usize, indices: usize },
// ... 기타 에러들
}
```

2. 복구 메커니즘

```rust
// consensus/proto_array/src/proto_array_fork_choice.rs

impl ProtoArrayForkChoice {

/// 모든 블록을 낙관적 상태로 재설정 (시작 시 복구용)

pub fn set_all_blocks_to_optimistic<E: EthSpec>(

&mut self,

spec: &ChainSpec,

) -> Result<(), String> {

for node_index in (0..self.proto_array.nodes.len()).rev() {

let node = self.proto_array.nodes.get_mut(node_index)

.ok_or("unreachable index out of bounds in proto_array nodes")?;


match node.execution_status {

ExecutionStatus::Invalid(block_hash) => {

node.execution_status = ExecutionStatus::Optimistic(block_hash);


// 가중치 복원

let mut restored_weight: u64 = self.votes.0

.iter()

.enumerate()

.filter_map(|(validator_index, vote)| {

if vote.current_root == node.root {

self.balances.effective_balances.get(validator_index)
```

```
} else {
None
}
})
.sum();

// 제안자 부스트 복원
if let Some(proposer_score_boost) = spec.proposer_score_boost {
if self.proto_array.previous_proposer_boost.root == node.root {
let proposer_score = calculate_committee_fraction::<E>(
&self.balances,
proposer_score_boost,
).ok_or("Failed to compute proposer boost")?;
self.proto_array.previous_proposer_boost.score = proposer_score;
restored_weight = restored_weight
.checked_add(proposer_score)
.ok_or("Overflow when adding boost to weight")?;
}
}

// 복원된 가중치를 노드와 조상들에게 적용
if restored_weight > 0 {
let mut node_or_ancestor = node;
```

```
loop {
node_or_ancestor.weight = node_or_ancestor.weight
.checked_add(restored_weight)
.ok_or("Overflow when adding weight to ancestor")?;


if  let Some(parent_index) = node_or_ancestor.parent {
node_or_ancestor = self.proto_array.nodes.get_mut(parent_index)
.ok_or(format!("Missing parent index: {}", parent_index))?;
} else {
break;
}
}
}
}
ExecutionStatus::Valid(block_hash) | ExecutionStatus::Optimistic(block_hash) =>
node.execution_status = ExecutionStatus::Optimistic(block_hash);
}
ExecutionStatus::Irrelevant(_) => (),
}
}


Ok(())
}
```

```
}
```

결론

Lighthouse의 Fork Choice 구현은 LMD-GHOST 알고리즘을 ProtoArray 데이터 구조를 통해 효율적으로 구현한 정교한 시스템입니다. 주요 특징은 다음과 같습니다:

핵심 설계 원칙

1. **효율성**: ProtoArray는 O(log n) 시간 복잡도로 헤드를 찾을 수 있습니다
2. **안전성**: 실행 페이로드 검증과 슬래싱 보호 메커니즘을 포함합니다
3. **확장성**: 엘라스틱 리스트와 배치 처리로 대규모 검증자를 지원합니다
4. **복구 가능성**: 시스템 재시작 시 상태를 복구할 수 있는 메커니즘을 제공합니다

주요 최적화 기법

1. **제안자 부스트**: 네트워크 지연에 대한 저항성 향상
2. **프루닝**: 메모리 사용량 관리
3. **배치 처리**: 증명 처리 성능 향상
4. **캐싱**: 자주 사용되는 값들의 캐싱

알고리즘 동작 흐름

1. **블록 추가**: 새 블록이 도착하면 검증 후 ProtoArray에 추가
2. **증명 처리**: 검증자의 투표를 VoteTracker에 기록
3. **헤드 계산**: 투표 변화량을 계산하고 가중치를 업데이트하여 최적의 헤드 결정
4. **재구성 검토**: 특정 조건 하에서 제안자 재구성 가능성 평가
5. **프루닝**: 최종화된 블록 이전의 오래된 블록들 제거

이러한 구현을 통해 Lighthouse는 Ethereum 2.0 네트워크에서 안정적이고 효율적인 포크 선택을 수행할 수 있으며, 네트워크의 합의 안전성과 활성성을 보장합니다.